

密码分析学实验报告

姓名：邱泽辉、江李阳、王健一

学号：202400460046、202400460042、202400460037

专业：密码科学与技术

日期：October 11, 2026

Contents

1 组员分工 **3**

2 实验环境 **3**

3 实验原理 **3**
3.1 差分路线 . . . 3
3.2 差分 . . . 3
3.3 CipherFour 算法 . . . 4
3.4 利用线性规划寻找差分路线 . . . 4
3.5 密钥恢复 . . . 5
3.5.1 信噪比 . . . 5
3.5.2 成功率 . . . 5

4 任务一 **6**
4.1 直接穷举思路 . . . 6
4.2 S 盒差分分布表 . . . 6
4.3 一轮差分传播 . . . 6
4.4 四轮动态规划 . . . 8
4.5 高概率差分的搜索 . . . 8
4.6 方法二：排除最优路线后的搜索 . . . 9
4.6.1 排除路线的依据 . . . 10
4.6.2 搜索结果 . . . 10

5 任务二 **11**

6 任务三 **12**
6.1 实验方法 . . . 12
6.2 实验结果 . . . 13

7 任务四 **14**
7.1 5 轮密钥恢复攻击 . . . 14
7.1.1 代码运行结果 . . . 15
7.1.2 复杂度分析 . . . 18

8 任务五 **18**
8.1 PHILOX-2W 的一轮差分传播 . . . 18
8.2 程序与参数 . . . 19
8.3 实验结果 . . . 19
8.4 与随机置换的对比 . . . 21

9 实验总结 **22**

A 实验代码 **23**

B 实验报告完整地址 **23**

C AI 使用情况 23

1 组员分工

- 江李阳：完成任务一、二、三的代码和对应实验报告章节内容的撰写。
- 王健一：完成任务四、五的代码和实验报告对应章节内容的撰写。
- 邱泽辉：完成任务一、二、三的代码撰写，实验报告最终检查润色。

2 实验环境

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约为 2.1 GHz；内存 16 GB。
- 软件环境：Windows、MacOS,Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。

3 实验原理

3.1 差分路线

i 轮差分路线是指对输入对 (X, X')，在 i 轮加密过程中满足对应第 j 轮的一对输出值 $(Y, Y')(1lej \leq i)$，满足 $\Delta X = X \oplus X' = \beta_0$，$\Delta Y = \oplus Y' = beta_j(1 \leq j \leq i)$，则称

$$\beta_0 \to \beta_1 \to \cdots \to \beta_i$$

为一条 i 轮差分路线。假设密码算法的输入 X 和各轮密钥 K_i 的取值是相互独立且均匀分布的，对于满足输入差分为 $beta_0$、输出差分为 β_i(显然满足输入差分为 β_0 的明文不只一对) 的 i 轮差分路线概率为 $DP(\beta_0 \to \beta_i)$, 其概率为各轮差分路线概率的乘积

$$\mathrm{DP}(\beta_0 \to \beta_1 \to \cdots \to \beta_i) = \prod_{(j=1)}^{i} \mathrm{DP}(\beta_{j-1} \to \beta_j)$$

注意这里差分路线是精确记录了包括起始、最终结果和中间每一种状态的一条路线。影响上述 i 轮差分路线的概率的主要因素是输入差分非零的 S 盒的个数以及输入差分非零的 S 盒对应的输出差分 (因为当前轮 S 盒的输出即为下一轮 S 盒的输入)，其中输入非零的 S 盒被称为活跃 S 盒。

3.2 差分

i 轮差分是指给定了输入差分 β_0 和 i 轮加密后的输出差分 β_i，满足上述条件的称为 i 轮差分。与 i 轮差分路线相比，它不指定加密的中间状态。因此，对于 i 轮差分的概率计算如下：假设满足输入差分为 β_0 且 i 轮之后输出差分为 β_i 的 i 轮差分路线共 s 条，则:

$$\begin{aligned}\mathrm{DP}(\beta_0 \xrightarrow{\text{i 轮}} \beta_i) &= \sum_{t=1}^{s} \mathrm{DP}(\beta_0 \xrightarrow{1\text{ 轮}} \beta_1^t \to \ldots \beta_{i-1}^t \xrightarrow{1\text{ 轮}} \beta_i) \\ &= \sum_{t=1}^{s} (\prod_{j=1}^{i} \mathrm{DP}(\beta_{j-1}^t \xrightarrow{1\text{ 轮}} \beta_j^t))\end{aligned}$$

也就是说 i 轮差分的概率是由所有满足输入和输出条件的 i 轮差分路线共同组成的，它的概率是要大于等于单条差分路线的概率的，而且想要精确计算差分的概率，还是要回到计算每一条差分路线的概率上去。

3.3 CipherFour 算法

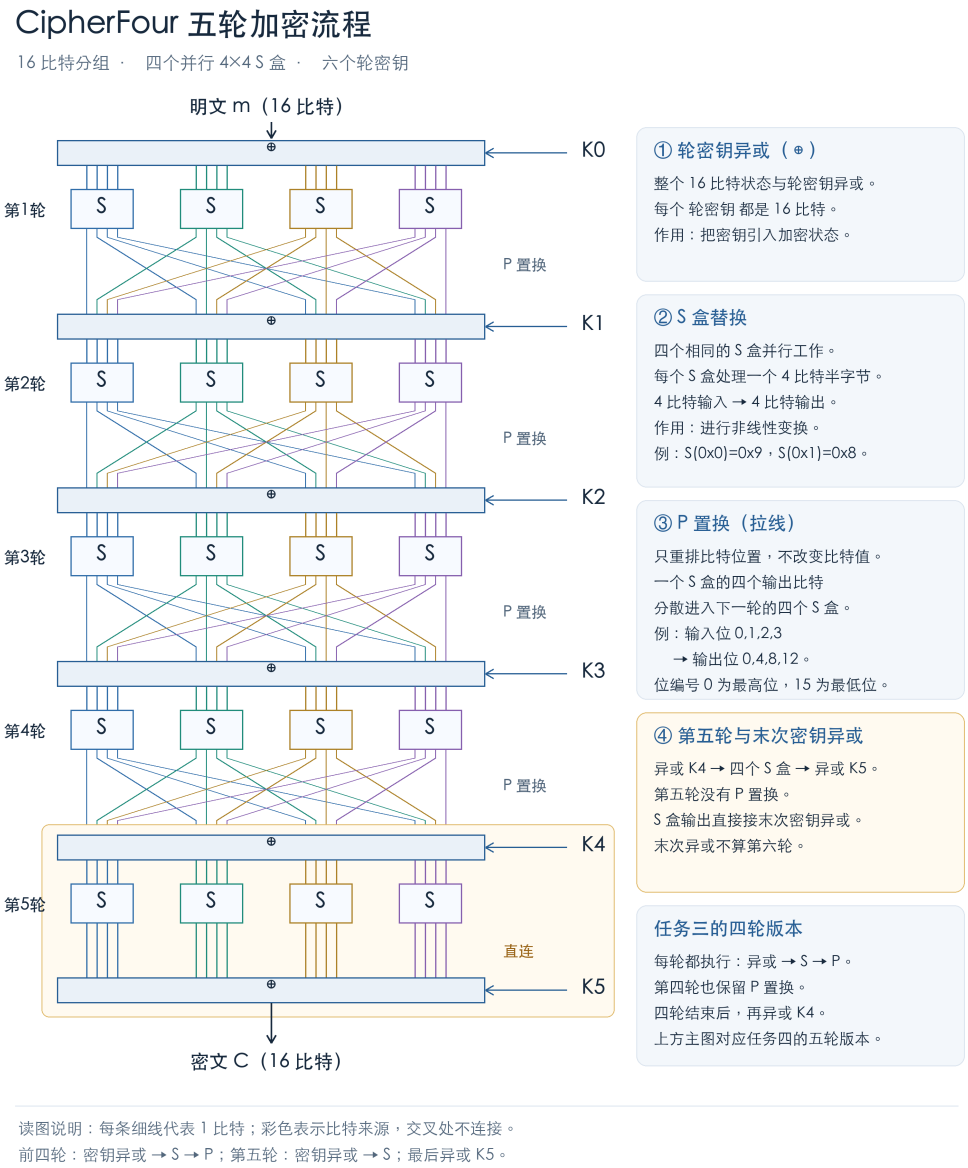


Figure 1: CipherFour 算法 5 轮加密流程图

3.4 利用线性规划寻找差分路线

定义 $B_r(\Delta)$ 为经过 r 轮后，到达差分状态 Δ 的所有路线中，概率最大的一条的概率。对于输入差分，把输入差分为 0 直接丢掉，因为输入差分为 0 之后的概率都是 1，只能

找到全 0 的路线，定义

$$B_0(\Delta) = \begin{cases} 1, & \Delta \neq 0, \\ 0, & \Delta = 0. \end{cases}$$

沿用书上的符号，假设每一轮的差分为 β_r
假设已经知道了 $B_{r-1}(\beta_{r-1})$,, 现在要计算 $B_r(\beta_r)$
从前驱 β_{r-1} 到达 β_r 的最佳完整路线概率为

$$\mathrm{Pr}(\beta_{r-1} \to \beta_r) = B_{r-1}(\beta_{r-1}) \cdot \mathrm{Pr}(\beta_{r-1}, \beta_r).$$

然后再遍历所有前驱，就有

$$B_r(\beta_r) = \max_{\beta_{r-1}} \{B_{r-1}(\beta_{r-1}) \cdot \mathrm{Pr}(\beta_{r-1}, \beta_r)\}$$

这在代码上看是比较简单的，只要遍历所有 β_{r-1} 就行，$B_{r-1}(\beta_{r-1}), Pr(\beta_{r-1}, \beta_r)$ 知道 β_{r-1} 后可以直接查表

但是理论上的东西，并不总是可行的，要知道每个 β_{r-1} 可是有 2^{16} 个，看上去也不是很多，但是注意这里的时间复杂度是在指数上的，也就是 $O(2^{16r})$

那么如果是 4 轮，就会达到 2^{64}, 那这就不是一个小数目了，所以要拆分成 4 个 S 盒

3.5 密钥恢复

3.5.1 信噪比

差分分析的密钥恢复阶段中，正确对个数与计数器的平均计数之比称为差分分析的信噪比，记为 S_N。设密钥恢复中的高概率差分概率为 p，可恢复的密钥长度为 k 比特，采样阶段选取 m 对明文，去噪后有 ϵm 对保存下来，恢复密钥阶段每个方程平均求得 η 个解，则:

$$S_N = \frac{mp}{\frac{\epsilon m \cdot \eta}{2^k}} = \frac{p \cdot 2^k}{\epsilon \cdot \eta}$$

3.5.2 成功率

成功率即攻击恢复出来的密钥是正确密钥的概率，记为 P_S。设差分分析中错误密钥的计数器相互独立且均匀分布，若差分概率为 p，可恢复的密钥长度为 k 比特，采样阶段选 m 对明文，信噪比为 S_N，计数器前 2^{k-a} 个为正确密钥的候选值，则对足够大的 k 和 m 有：

$$P_S = \Phi(\frac{\sqrt{pmS_N} - \Phi^{-1}(1 - 2^{-a})}{\sqrt{S_N + 1}})$$

其中 Φ 为标准正态分布函数。在以上假设下，要保证成功率为 P_S，需要的明文对为:

$$m = \frac{(\sqrt{S_N + 1}\Phi^{-1}(P_S) + \Phi^{-1}(1 - 2^{-a}))^2}{S_N} p^{-1}$$

4 任务一

本任务要求寻找除题目给定差分之外的高概率四轮差分。题目给出的差分为

$$(0x0, 0x0, 0x0, 0x1) \xrightarrow{4\text{轮}} (0x0, 0x0, 0x0, 0x1),$$

也可以写成

$$0x0001 \xrightarrow{4\text{轮}} 0x0001.$$

4.1 直接穷举思路

对于固定的输入差分 α，可以遍历所有 2^{16} 个 16 比特明文 x，构造有序明文对 $(x, x \oplus \alpha)$，然后分别进行四轮 CipherFour 加密。若最终密文差分为 β，则将对应计数加一。因此，固定输入、输出差分的概率为

$$P(\alpha \to \beta) = \frac{\#\{x : E(x) \oplus E(x \oplus \alpha) = \beta\}}{2^{16}}.$$

对于每一个非零输入差分，共有 2^{16} 个有序明文对；如果不区分明文对的顺序，则有 2^{15} 个无序明文对。若对全部 $2^{16} - 1$ 个非零输入差分都进行穷举，需要处理约 2^{32} 组明文对，计算开销较大。因此，程序使用 S 盒差分分布表（DDT）对差分进行动态规划。

4.2 S 盒差分分布表

CipherFour 使用一个 4 进 4 出的 S 盒。对于输入差分 α 和输出差分 β，DDT 定义为

$$\mathrm{DDT}[\alpha][\beta] = \#\left\{x \in \{0, 1, \ldots, 15\} : S(x) \oplus S(x \oplus \alpha) = \beta\right\}.$$

因此，单个 S 盒的差分传播概率为

$$T[\alpha][\beta] = \frac{\mathrm{DDT}[\alpha][\beta]}{16} = \frac{\mathrm{DDT}[\alpha][\beta]}{2^4}.$$

代码中的 DDT 计算如下：

```
for diff_in in range(16):
    for value in range(16):
        diff_out = S[value] ^ S[value ^ diff_in]
        diff_table[diff_in][diff_out] += 1
```

其中，`diff_table[diff_in][diff_out]` 保存输入差分为 `diff_in`、输出差分为 `diff_out` 的出现次数。

4.3 一轮差分传播

一个 16 比特差分可以拆成四个半字节：

$$\Delta = (a_0, a_1, a_2, a_3).$$

若四个 S 盒的输出差分为

$$\Gamma = (b_0, b_1, b_2, b_3),$$

则在固定当前差分的条件下，四个 S 盒同时发生该传播的概率为

$$Q(\Delta, \Gamma) = \prod_{i=0}^{3} T[a_i][b_i].$$

例如，输入差分 $0x0001 = (0, 0, 0, 1)$ 时，前三个 S 盒的输入差分为 0，并且

$$T[0][0] = 1.$$

只有最后一个 S 盒产生分支。对于本实验使用的 S 盒，输入差分 1 可以传播为输出差分 1、2、5，其概率分别为 1/2、1/4、1/4。因此，经过 S 盒层后可能得到

$$0x0001\ (1/2), \qquad 0x0002\ (1/4), \qquad 0x0005\ (1/4).$$

代码中的 `score` 保存四个半字节的联合概率分布，其形状为

$$(\text{batch}, 16, 16, 16, 16).$$

处理某一个半字节时，程序计算

$$F_{\text{new}}(\beta) = \sum_{\alpha=0}^{15} F_{\text{old}}(\alpha) T[\alpha][\beta].$$

这对应代码中的矩阵乘法：

```
score_matrix = moved_score.reshape(-1, 16)
updated_score = score_matrix @ probability_table
```

程序依次处理四个 S 盒：

```
for position in range(4):
    score = update_one_sbox_distribution(
        score, probability_table, position
    )
```

对于一个确定的当前 16 比特差分，这一过程等价于四个 S 盒概率的乘积；当当前状态本身是多个差分状态的概率混合时，程序还会对所有前驱状态的概率进行求和，因此需要保存四个半字节的联合分布，而不能只分别保存四个边缘概率。

四个 S 盒处理完成后，将得到的 16 比特差分经过 P 置换。代码预先计算所有 16 比特状态的置换结果：

```
permutation = np.array(
    [P_box(value) for value in range(STATE_SIZE)],
    dtype=np.int32,
)
```

由于 P 是比特置换，满足

$$P(X) \oplus P(Y) = P(X \oplus Y),$$

因此可以直接对差分状态进行 P 置换。经过 P 置换后的差分分布作为下一轮的输入。

4.4 四轮动态规划

设初始输入差分为 α，经过 r 轮后的差分分布为 $D_r^{(\alpha)}$。初始时只有输入差分本身的概率为 1：

$$D_0^{(\alpha)}(\Delta) = \begin{cases} 1, & \Delta = \alpha, \\ 0, & \Delta \neq \alpha. \end{cases}$$

代码通过以下语句完成初始化：

```
score[rows, nibble0, nibble1, nibble2, nibble3] = 1.0
```

记一轮四个 S 盒和 P 置换的差分转移概率为 $R(\Delta, \Delta')$，则每轮动态规划满足

$$D_{r+1}^{(\alpha)}(\Delta') = \sum_{\Delta} D_r^{(\alpha)}(\Delta) R(\Delta, \Delta').$$

因此，四轮后的输出差分概率为

$$D_4^{(\alpha)}(\beta) = \sum_{\Delta_1, \Delta_2, \Delta_3} R(\alpha, \Delta_1) R(\Delta_1, \Delta_2) R(\Delta_2, \Delta_3) R(\Delta_3, \beta).$$

这个求和包含了从输入差分 α 到输出差分 β 的所有差分路线。代码的外层循环对应四轮传播：

```
for _ in range(NUMBER_OF_ROUNDS):
    for position in range(4):
        score = update_one_sbox_distribution(
            score, probability_table, position
        )

    after_sbox = score.reshape(batch_size, STATE_SIZE)
    after_pbox[:, permutation] = after_sbox
    score = after_pbox.reshape(
        batch_size, 16, 16, 16, 16
    )
```

这里每一轮的输入和输出都是 16 比特差分状态的概率分布，不是具体的明文或密文。四轮循环结束后，score 中保存每个输入差分经过四轮后得到所有输出差分的概率。

4.5 高概率差分的搜索

程序从 0x0001 到 0xFFFF 遍历全部 $2^{16} - 1$ 个非零输入差分，并将输入差分按 32 个一批进行处理，以降低内存开销。分批处理不会减少搜索范围。

对于每个输入差分 α，四轮动态规划得到所有输出差分 β 的概率分布。程序使用

```
output_differences = np.argmax(
    output_distribution, axis=1
)
```

找到该输入差分对应的最大概率输出差分。随后，再在所有输入差分对应的候选结果中选择全局最大值。

题目要求排除 $0x0001 \to 0x0001$，因此代码只对输入差分 0x0001 的输出分布执行：

```
candidate_distribution[
    EXCLUDED_OUTPUT_DIFFERENCE
] = 0.0
```

然后重新选择该输入差分对应的最佳输出差分。

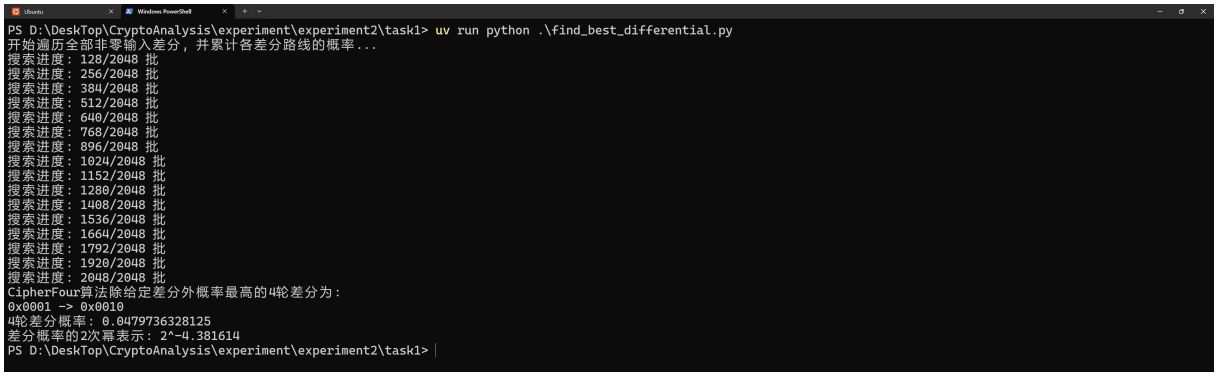


Figure 2: 寻找高概率差分

程序最终找到的四轮高概率差分为

$$(0x0, 0x0, 0x0, 0x1) \longrightarrow (0x0, 0x0, 0x1, 0x0),$$

即

$$0x0001 \longrightarrow 0x0010.$$

其基于 DDT 动态规划得到的四轮差分概率为

$$P(0x0001 \to 0x0010) = 0.0479736328125 \approx 2^{-4.381614}.$$

这里计算的是连接相同输入、输出差分的所有差分路线的概率总和，而不是某一条单独差分路线的概率。后续任务二再通过枚举具体明文对，统计该输入、输出差分在固定 CipherFour 置换网络中的实际路线和频率。

4.6 方法二：排除最优路线后的搜索

示例程序搜索的是概率最高的单条四轮差分路线。设 Δ_0 为输入差分，Δ_r 为第 r 轮经过 S 盒和 P 置换后的差分，则一条路线写为

$$\Delta_0 \to \Delta_1 \to \Delta_2 \to \Delta_3 \to \Delta_4.$$

程序得到的最优路线为

$$0\text{x}0001 \to 0\text{x}0001 \to 0\text{x}0001 \to 0\text{x}0001 \to 0\text{x}0001,$$

每轮传播概率均为 $1/2$，所以该路线的概率为 $(1/2)^4 = 1/16$。为了判断是否存在概率相同的其他路线，需要在搜索中排除这条已知路线。

4.6.1 排除路线的依据

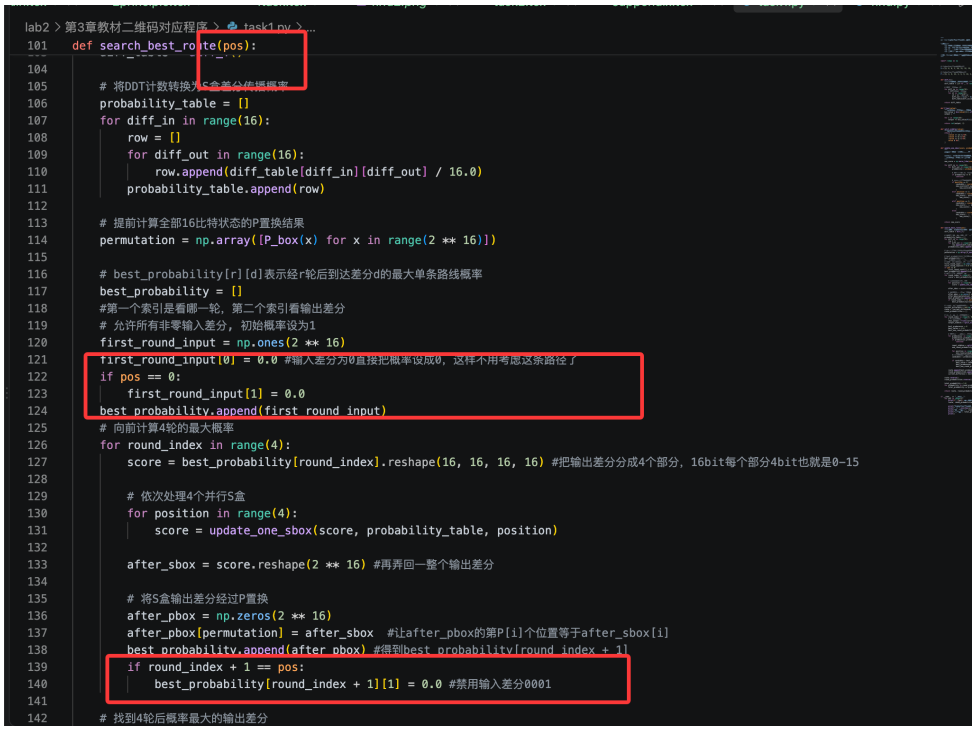
设 $B_r(\Delta)$ 表示经过 r 轮后到达差分状态 Δ 的最大单条路线概率。一轮差分转移概率仍记为 $R(u,v)$，则示例程序的更新规则可写为

$$B_{r+1}(v) = \max_{u} \{ B_r(u) R(u,v) \}.$$

这里取最大值，与方法一中对所有前驱路线求和不同。初始化时，所有非零输入差分的 B_0 均设为 1，而 $B_0(0) = 0$。这是因为零输入差分会以概率 1 沿全零路线传播；将其排除，才能搜索非零差分路线。

原最优路线在 $\Delta_0, \ldots, \Delta_4$ 的五个位置上均为 0x0001。任意另一条路线至少有一个位置与之不同，因此可以分别在位置 $j = 0, 1, 2, 3, 4$ 禁用差分 0x0001，对五次搜索结果取最大概率。五个受限搜索覆盖了所有不同于原最优路线的路线；某条路线若在多个位置不同，也可能在多次搜索中重复出现。

实现时，$j = 0$ 对应将初始概率数组的 0x0001 项置零；$j = 1, \ldots, 4$ 对应在第 j 轮完成 S 盒和 P 置换后，将 $B_j(0\text{x}0001)$ 置零。每次四轮搜索结束后，再由最终状态逆向追溯各轮差分，得到完整路线。图 3 展示了代码中的位置参数和禁用操作。



```
lab2 > 第3章教材二维码对应程序 > task1.py > ...
101  def search_best_route(pos):
104
105      # 将DDT计数转换为S盒差分传播概率
106      probability_table = []
107      for diff_in in range(16):
108          row = []
109          for diff_out in range(16):
110              row.append(diff_table[diff_in][diff_out] / 16.0)
111          probability_table.append(row)
112
113      # 提前计算全部16比特状态的P置换结果
114      permutation = np.array([P_box(x) for x in range(2 ** 16)])
115
116      # best_probability[r][d]表示经r轮后到达差分d的最大单条路线概率
117      best_probability = []
118      #第一个索引是看哪一轮，第二个索引看输出差分
119      # 允许所有非零输入差分，初始概率设为1
120      first_round_input = np.ones(2 ** 16)
121      first_round_input[0] = 0.0 #输入差分为0直接把概率设成0，这样不用考虑这条路径了
122      if pos == 0:
123          first_round_input[1] = 0.0
124      best_probability.append(first_round_input)
125      # 向前计算4轮的最大概率
126      for round_index in range(4):
127          score = best_probability[round_index].reshape(16, 16, 16, 16) #把输出差分分成4个部分，16bit每个部分4bit也就是0-15
128
129          # 依次处理4个并行S盒
130          for position in range(4):
131              score = update_one_sbox(score, probability_table, position)
132
133          after_sbox = score.reshape(2 ** 16) #再弄回一整个输出差分
134
135          # 将S盒输出差分经过P置换
136          after_pbox = np.zeros(2 ** 16)
137          after_pbox[permutation] = after_sbox  #让after_pbox的第P[i]个位置等于after_sbox[i]
138          best_probability.append(after_pbox) #得到best_probability[round_index + 1]
139          if round_index + 1 == pos:
140              best_probability[round_index + 1][1] = 0.0 #禁用输入差分0001
141
142      # 找到4轮后概率最大的输出差分
```

Figure 3: 在指定位置排除已知最优路线的差分状态

4.6.2 搜索结果

图 4 给出五次受限搜索的输出。表 1 将各路线整理为十六进制差分状态序列，并按四轮概率的乘积给出路线概率。

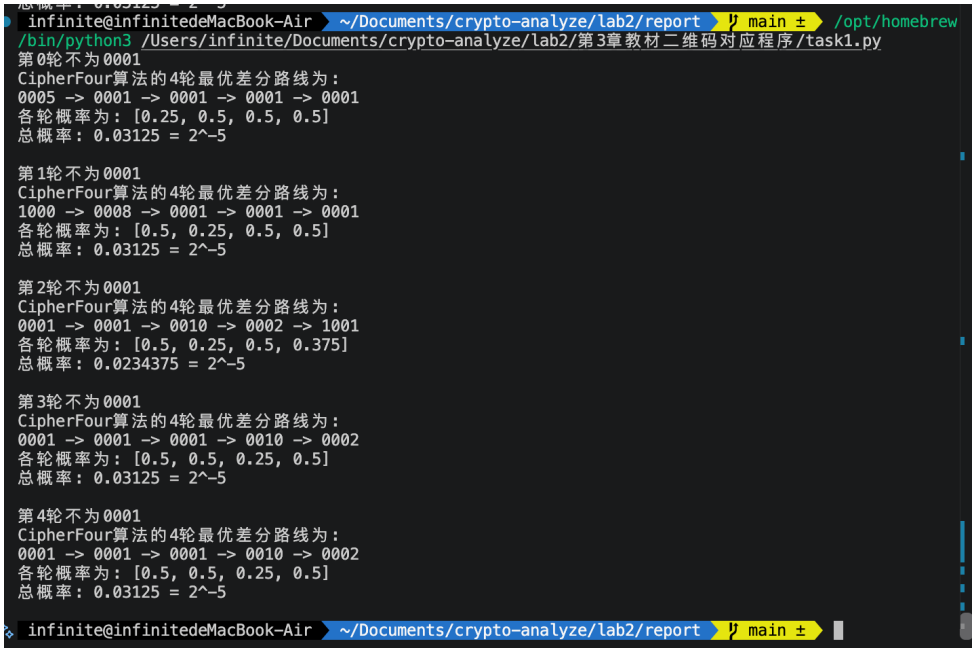


Figure 4: 分别禁用五个位置后的差分路线搜索结果

Table 1: 排除已知最优路线后得到的差分路线

禁用位置 j	路线概率	差分路线（十六进制）
0	$1/32$	$0005 \to 0001 \to 0001 \to 0001 \to 0001$
1	$1/32$	$1000 \to 0008 \to 0001 \to 0001 \to 0001$
2	$3/128$	$0001 \to 0001 \to 0010 \to 0002 \to 1001$
3	$1/32$	$0001 \to 0001 \to 0001 \to 0010 \to 0002$
4	$1/32$	$0001 \to 0001 \to 0001 \to 0010 \to 0002$

在禁用位置 $j = 2$ 的一次搜索中，四轮概率分别为 $1/2, 1/4, 1/2, 3/8$，实际乘积为 $3/128 = 0.0234375$。截图末尾的 2^{-5} 是程序对概率取对数后再转为整数的显示结果，不能作为这条路线的精确概率。其余四次受限搜索均得到 $1/32$。因此，排除原最优路线后，概率最高的单条路线为 $1/32$，低于原路线的 $1/16$；位置 $j = 3$ 与 $j = 4$ 搜索到了同一条路线，五次结果共对应四条不同路线。

这些结果反映的是单条差分路线的概率。若只看输入、输出差分，上述四条路线分别对应 $0x0005 \to 0x0001$、$0x1000 \to 0x0001$、$0x0001 \to 0x1001$ 和 $0x0001 \to 0x0002$。同一输入、输出差分还可能由其他路线实现，因此这些路线概率不能直接当作对应差分的总概率；差分总概率应按方法一对所有路线求和。

5 任务二

根据任务一找到的新 4 轮高概率差分，要寻找所有对应的差分路线，思路为遍历所有 2^{16} 种输入差分对，即 $x \in \{0, 1, \ldots 2^{16} - 1\}$、$x' = x \oplus \Delta_{in}$，记录下 4 轮加密过程中每一轮的中间异或结果，当满足输出差分 $y \oplus y' = \Delta_{out}$ 时，该路线即为所要求的 4 轮差分

的差分路线，将该路线记录下来，最后输出所有差分路线并计算概率 (注意这里按照输入是有序对时共 2^{16} 种满足条件的差分对，若按照无序对考察，则差分路线和总的差分对数量均减半，但最终概率不变)，计算概率为:

$$P = \frac{\#\{x : F^4(x) \oplus F^4(x \oplus \Delta_{in}) = \Delta_{out}\}}{2^{16}}$$

按照上述思路编写代码，运行结果如下:

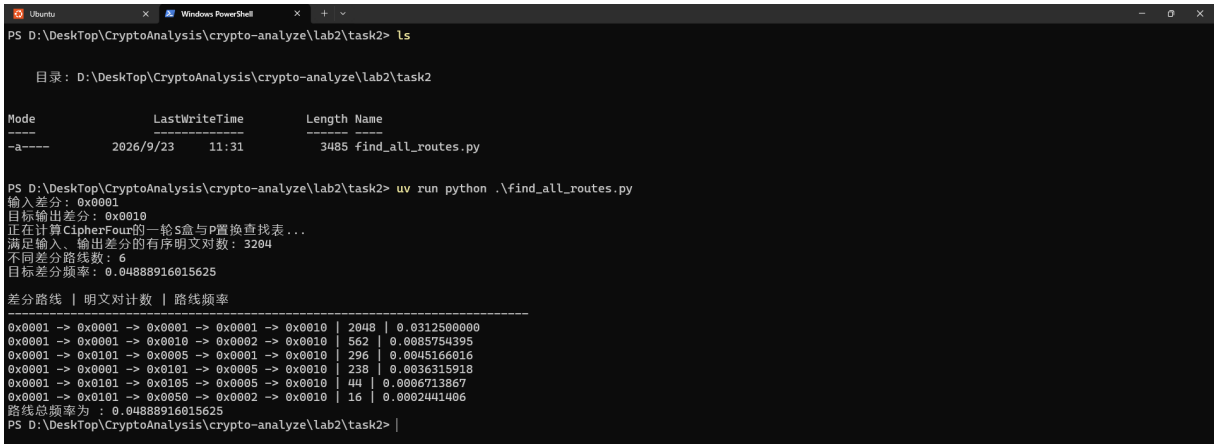


Figure 5: 所有新 4 轮差分路线

6 任务三

本任务使用五组轮密钥，检验四轮 CipherFour 的差分概率在具体密钥下是否保持不变。输入差分和目标输出差分分别为

$$\Delta_{in} = 0x0001, \qquad \Delta_{out} = 0x0010.$$

每组密钥包含五个 16 比特字：前四个分别在四轮开始时与状态异或，第五个在四轮结束后与状态异或。最后一个密钥字是输出白化密钥，不是第五个 S-P 轮。

6.1 实验方法

程序先计算 16 比特状态经过一层 S 盒和 P 置换的查找表。对任一输入状态 v，查找表保存

$$SP[v] = P(S(v)).$$

因此，对固定密钥组 $(k_0, k_1, k_2, k_3, k_4)$，四轮加密可写为

$$X_0 = x, \qquad X_{i+1} = SP[X_i \oplus k_i] \quad (i = 0, 1, 2, 3), \qquad E_k(x) = X_4 \oplus k_4.$$

代码先为全部 2^{16} 个状态构造查找表：

```
for value in range(STATE_SIZE):
    SP_table[value] = P_box(S_box(value))
```

再用该表执行四轮加密：

```
state = plainText
for round_index in range(NUMBER_OF_ROUNDS):
    state = SP_table[state ^ keys[round_index]]
return state ^ keys[NUMBER_OF_ROUNDS]
```

实验固定随机种子为 20260922，随机生成五组互不相同的密钥。对每组密钥，程序计算全部 2^{16} 个明文的密文，并枚举

$$(x, x') = (x, x \oplus 0x0001), \qquad 0 \le x < 2^{16}.$$

若密文差分为 $0x0010$，就将匹配数加一。对第 j 组密钥，记匹配数为 N_j，则实验频率为

$$P_j = \frac{N_j}{2^{16}}.$$

循环遍历所有 x，所以每个无序明文对按正、反两个顺序各统计一次。

6.2 实验结果

五组密钥及其统计结果如下。表中密钥字按 $(k_0, k_1, k_2, k_3, k_4)$ 排列，其中 k_4 是四轮后的输出白化密钥。

Table 2: 五组密钥下四轮差分 $0x0001 \to 0x0010$ 的统计结果

组别	密钥组 $(k_0, k_1, k_2, k_3, k_4)$	匹配对数 N_j	频率 P_j
1	[0xB8E5, 0x3463, 0x03F1, 0x26A1, 0x13E1]	3080	0.0469970703
2	[0x216F, 0xEAC8, 0xD44F, 0x1EE4, 0x6EA1]	3058	0.0466613770
3	[0x4DEC, 0x374E, 0x68DC, 0xC60D, 0x57F9]	3076	0.0469360352
4	[0x945E, 0x835B, 0x9F29, 0x33D3, 0xC82A]	3054	0.0466003418
5	[0xBC38, 0x20B4, 0x3ACF, 0x3CE6, 0xD29B]	3070	0.0468444824

五组频率的平均值为

$$\overline{P} = \frac{1}{5}\sum_{j=1}^{5} P_j = 0.0468078613.$$

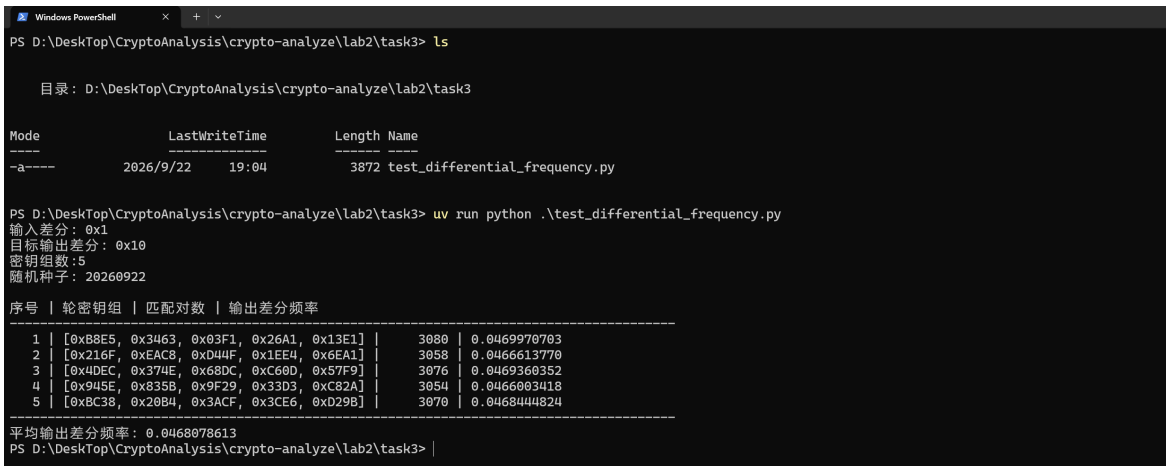


Figure 6: 五组密钥下的差分频率统计过程与结果

五组频率落在 0.04660 到 0.04700 之间，平均约为 0.04681。任务一通过 DDT 逐轮传播差分概率，得到 0.0479736328125；本任务则对每组固定密钥穷举全部明文对，得到该具体密钥下的精确频率。两者接近但不完全相等，说明固定密钥下的逐轮差分传播并不等同于 DDT 模型给出的概率。该结果与任务一中的 DDT 差分概率传播模型、任务二中找到所有 4 轮差分路线的概率大致等，说明实际运行结果与理论分析非常相近，相互印证了前面几个实验的结果是正确的。

四轮结束后的白化密钥 k_4 对一对密文都执行相同的异或，因此在密文差分中抵消。频率随密钥组变化，来自前四轮密钥对各轮 S 盒输入状态的影响。

7 任务四

7.1 5 轮密钥恢复攻击

根据第 1) 问得到的 4 轮高概率差分路线：

$$\text{0x0001} \longrightarrow \text{0x0010}$$

阐述 5 轮密钥恢复攻击的基本过程：这里采用 4+1 轮模式，即在 4 轮高概率差分的末尾添加一轮。其中选择的明文对 (P, P') 满足

$$P \oplus P' = \Delta_{in} = \text{0x0001}$$

由高概率差分可知，四轮加密后相应输出 (C_4, C_4') 满足

$$C_4 \oplus C_4' = \Delta_{out} = \text{0x0010}$$

概率为 0.049，远大于随机情况的 $1/2^{16}$。探测其中第四轮加密结果这一中间情况的方法就是猜测第五轮的轮密钥 k_5：

- 若猜对 k_5，则由五轮加密的到的密文 (C, C') 解密回正确的 (C_4, C_4')，满足 $C_4 \oplus C_4' = \text{0x0010}$ 的概率为高概率的 0.049；
- 若猜错，则计算出来的 (C_4, C_4') 是错误的/随机的，以随机概率满足，即 $\Pr(C_4 \oplus C_4' = \text{0x0010}) = \frac{1}{2^{16}}$。

因此基本密钥恢复方法为随机选择 m 对满足输入差分的明文对，对 k_5 的每一种可能设置一个计数器，满足:

$$S^{-1}(C \oplus k_5) \oplus S^{-1}(C' \oplus k_5) = \Delta_{out}$$

的则令计数器值 +1，处理完 m 对明文后，正确密钥的计数将明显高于错误密钥的计数，从而恢复出正确的密钥。

但是我们注意到，并非所有输入对 (P, P') 都能满足在四轮加密后满足 $C_4 \oplus C_4' = \Delta_{out}$，这样不满足的对称为错误对，恢复密钥攻击实际要使用的是正确对，错误对会对密钥恢复造成干扰，有以下事实：

- 正确对 + 正确密钥一定满足 $C_4 \oplus C_4' = \Delta_{out}$；
- 正确对 + 错误密钥以随机概率满足 $C_4 \oplus C_4' = \Delta_{out}$；
- 错误对 + 正确密钥一定不满足上式，即 $C_4 \oplus C_4' \neq \Delta_{out}$；

- 错误对 + 错误密钥以随机概率满足 $C_4 \oplus C_4' = \Delta_{out}$。

因此结合采样、去噪、恢复密钥三个步骤来筛选正确对，提高密钥恢复的成功概率。

- 采样：随机选择明文 P，计算 $P' = P \oplus (0x0001)$，重复该过程 m 次，得到 m 对满足输入差分的明文对，做 5 轮 CipherFour 算法加密得到对应的密文对。我们期望得到/留下的是正确对，满足 $C_4 \oplus C_4' = 0x0010$，其中发现过 S 盒子的输入 0 差分，输出还是 0 差分；且查过 S 盒的 DDT 后发现输入差分为 1 时，输出差分只可能是 $1, 2, 5$, 因此 5 轮加密后正确对应满足:

$$\begin{cases} \Delta C_{5,0} = \Delta C_{5,1} = \Delta C_{5,3} = 0 \\ \Delta C_{5,2} = 0x1/0x2/0x5 \end{cases}$$

- 去噪：按照上面的分析，对 m 对密文，计算器差分值 ΔC_5，保留满足:

$$\Delta C_5 \in \{0x0, 0x0, h, 0x0 | h \in \{0x1, 0x2, 0x5\}\}$$

  注意这里保留的逻辑是，正确对一定满足上式所以被保留，特殊错误对 (比如四轮加密中间值其他 (第 1，2，4) 字节差分为 0、第 3 半字差分并非为 0x1 的错误对也满足过 S 盒后为 h) 能满足上式而被保留，大部分错误对被排除掉，因此去噪后并不是所有留下的都是正确对，但能显著提高正确对的占比。

- 恢复密钥：设去噪后有 ϵn 对密文保留下来，记最后一轮第三个 S 盒相应的密钥为 $k_{5,2}$，我们的目标就是恢复这一 4bit 密钥（实际上由于其他 3 个 S 盒的输入差分都为 0 差分，因此任何密钥都能满足输入零差分到输出零差分，缺少有效信息恢复密钥其他比特），对每一对密文信息建立共 ϵn 个下述方程:

$$S^{-1}(C_{5,2}) \oplus S^{-1}(C_{5,2}' \oplus k_{5,2}) = 0x1$$

  按照前面的密钥计数器方法，保留前 2^{4-a} 个作为正确密钥的候选值。对于上述方程的求解，不妨转成正向方程来看:

$$S(x \oplus k) \oplus S(x' \oplus k) = \beta$$

  其中 S 为过 S 盒查表运算，(x, x', β) 已知 (在这里 (x, x') 就是四轮加密中间值，β 就是五轮加密后最终密文对的差分)。一种方法是穷举 k 的所有可能，代入方程进行验证；另一种方法为借助 S 盒的差分分布表求解：记 $\oplus x' = \alpha$，S 盒的所有满足输入差分为 α、输出差分为 β 的可能输入集合为 $IN_S(\alpha, \beta)$，得到的候选密钥集合为 $\{x \oplus X | X \in IN_S(\alpha, \beta)\}$

7.1.1 代码运行结果

首先运行明密文文件验证程序。使用实验指导书给出的六个密钥字

$$(0x0123, 0x4567, 0x89AB, 0xCDEF, 0xFEDC, 0xBA98)$$

这里先用上述六个轮密钥重新加密文件中的 2^{16} 个明文，计算发现密文与文件中的密文全部匹配。随后将最后一个密钥字的最后一个半字节由 0x8 改为 0x9，得到 $K_6 = 0xBA99$，此时 2^{16} 个密文全部不匹配。这个对照确认了本地文件确实对应指导书给出的密钥组和当前采用的加密结构，为后续筛选及密钥枚举提供了匹配的实验数据。



Figure 7: 验证指导书密钥与明密文文件的对应关系

在已验证的明密文文件上运行密文筛选和半字节密钥枚举程序。筛选后保留 4028 对，去噪前正确对占比约 4.8%，去噪后占比

$$\frac{2^{16} \times 0.048}{4028} \approx 78\%$$

正确对占比显著提高。程序对目标半字节的 $2^4 = 16$ 种取值逐一计数。根据给定密钥 $K_6 = 0\text{x}BA98$，目标半字节为 0x9。它的计数达到最高值 3172，与错误候选 0x8 并列。因此，本次枚举将正确的 4 bit 半字节包含在最高分候选中；当前计数结果还不能在 0x8 和 0x9 之间作出唯一判断。候选计数如下：

Table 3: 恢复目标半字节的候选计数

半字节候选	计数
0x8, 0x9	3172
0xC, 0xD	1902
0x0, 0x1	1544
0xA, 0xB	1468
0x2, 0x3	1268
0x4, 0x5	1232
0x6, 0x7	956
0xE, 0xF	198

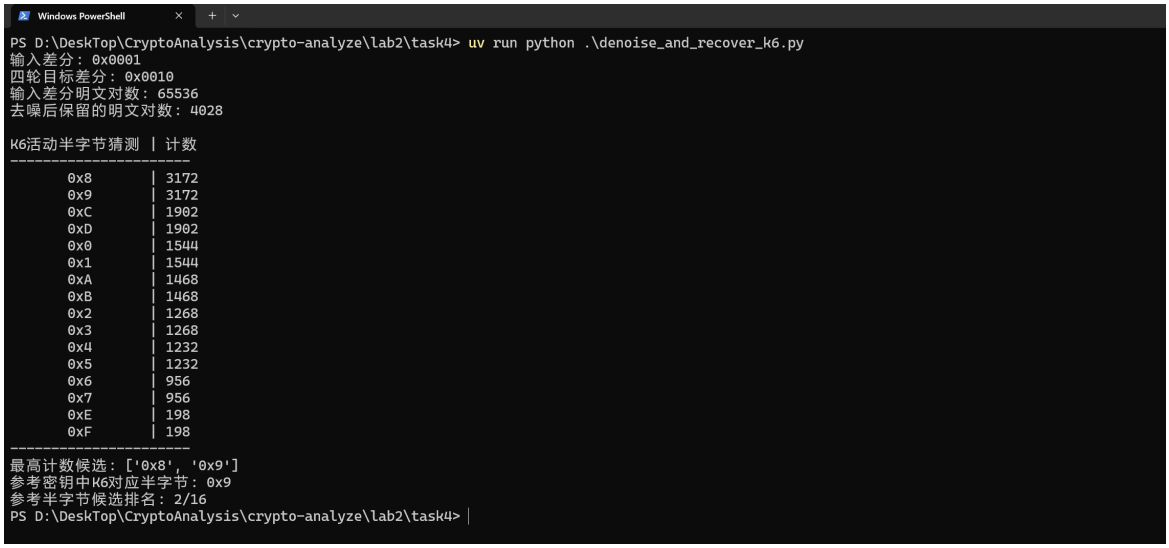


Figure 8: 筛选后对目标半字节进行差分计数的结果

随后对五组新生成的六字密钥重复筛选实验。程序复用文件中的全部明文，但为每组六字密钥分别重新计算对应的五轮密文。任务一得到的四轮差分理论概率为 $p = 0.0479736328125$；当 $m = 2^{16}$ 时，理论正确对数为

$$mp = 2^{16} \times 0.0479736328125 = 3144.$$

设某组密钥下实际筛选保留 N_j 对，则用 mp/N_j 估计保留对中正确对所占比例。五组统计结果如下：

Table 4: 五组新密钥下的筛选保留数及正确对占比估计

组别	六字密钥组 $(K_1, \ldots, K_6)$	实际保留数 N_j	mp/N_j
1	[0x0E6B, 0x0286, 0xAD7C, 0x496C, 0xFEDE, 0x93AE]	3906	80.49%
2	[0x36B5, 0xC640, 0x98AB, 0x1A0C, 0x095F, 0xF3C0]	3702	84.93%
3	[0xC0E4, 0x3F92, 0x4112, 0x1D08, 0x734C, 0xFB64]	3836	81.96%
4	[0x49A0, 0x7650, 0x9B9A, 0xAE68, 0xAA6B, 0x1255]	3696	85.06%
5	[0x8108, 0x5E1F, 0x13DD, 0xA3D8, 0xE75D, 0xB84D]	3656	86.00%

五组实际保留数的平均值为 3759.20 对，通过去噪阶段后正确对的平均占比为

$$\frac{2^{16} \times 0.048}{3759.20} \approx 84\%$$

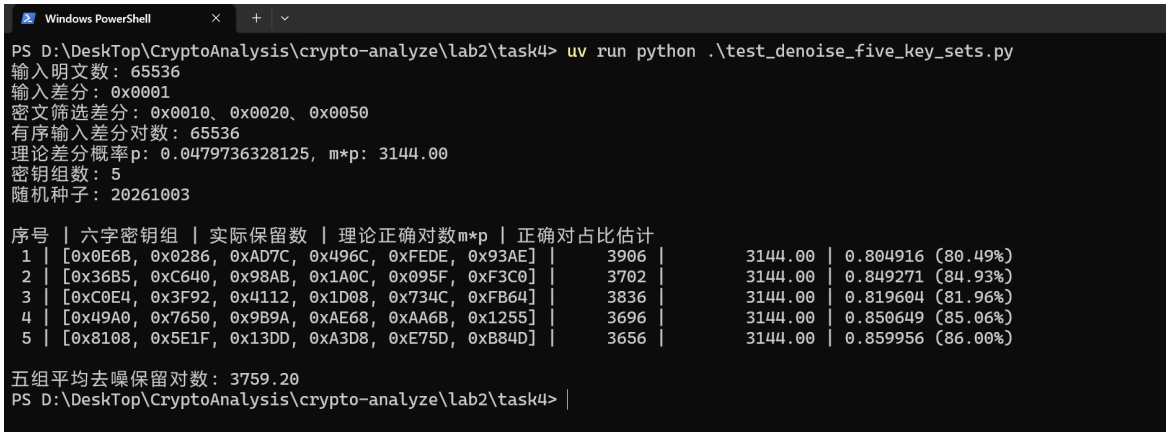


Figure 9: 五组新密钥下的去噪保留数及正确对占比估计

7.1.2 复杂度分析

- 数据复杂度：选择了 2^{16} 个明文对，即 2^{17} 个明文。
- 时间复杂度：
  - 采样阶段要获得 2^{17} 个明文对应的密文，即做 2^{17} 次 5 轮 CipherFour 算法的加密运算；
  - 去噪阶段计算 2^{16} 个密文对的差分值，即 2^{16} 次异或运算；
  - 恢复密钥阶段：去噪后保留 ϵn 对密文，每个 4 bit 半字节有 2^4 种猜测。对每一种猜测，都需要逐一检查全部保留对，因此需要 $2^4 \times \epsilon n$ 次差分方程验证。本次保留 4028 对，对应 $16 \times 4028 = 64448$ 次验证。
- 存储复杂度：
  - 采样阶段存储选择明文对应的密文，即 $2^{17} \times 16 = 2^2$1bit $= 256$KB；
  - 4bit 每种设置计数器，若每个计数器占 8bit，则计数器的存储复杂度约为 $2^4 \times 8 = 2^7$bit。

8 任务五

8.1 PHILOX-2W 的一轮差分传播

按照实验指导书中的公式，定义

$$\mathrm{mulhi}_M(X) = \left\lfloor \frac{X \cdot M}{2^W} \right\rfloor, \qquad \mathrm{mullo}_M(X) = (X \cdot M) \bmod 2^W.$$

其中每个分支为 W 比特，M 为固定常数，K 为一轮子密钥。PHILOX-2W 的一轮变换为

$$L' = \mathrm{mulhi}_M(R), \qquad R' = \mathrm{mullo}_M(R) \oplus L \oplus K.$$

设一对输入为 (L_1, R_1) 和 (L_2, R_2)，其输入差分为

$$\Delta L = L_1 \oplus L_2, \qquad \Delta R = R_1 \oplus R_2.$$

因此可令 $L_2 = L_1 \oplus \Delta L$、$R_2 = R_1 \oplus \Delta R$。经过一轮变换后，左分支输出差分为

$$\begin{aligned}\Delta L' &= L_1' \oplus L_2' \\ &= \mathrm{mulhi}_M(R_1) \oplus \mathrm{mulhi}_M(R_1 \oplus \Delta R),\end{aligned}$$

右分支输出差分为

$$\begin{aligned}\Delta R' &= R_1' \oplus R_2' \\ &= \mathrm{mullo}_M(R_1) \oplus L_1 \oplus K \oplus \mathrm{mullo}_M(R_1 \oplus \Delta R) \oplus L_1 \oplus \Delta L \oplus K \\ &= \mathrm{mullo}_M(R_1) \oplus \mathrm{mullo}_M(R_1 \oplus \Delta R) \oplus \Delta L.\end{aligned}$$

可见，相同的轮密钥 K 在差分中抵消。给定输入差分后，输出差分只需遍历 R_1 的取值即可确定；对每个 R_1，所有 2^W 个 L_1 都产生相同的输出差分。当 $\Delta R = 0$ 时，两次乘法输入相同，乘积高、低字的差分均为零，因此

$$(\Delta L, 0) \longrightarrow (0, \Delta L)$$

以概率 1 发生。特别地，输入差分 $(\texttt{0x01}, \texttt{0x00})$ 必然传播为输出差分 $(\texttt{0x00}, \texttt{0x01})$。

8.2 程序与参数

程序按照上述一轮公式实现 mulhi_M、mullo_M 和轮变换，并对全部 $2^{2W} = 2^{16}$ 个输入状态 (L, R) 进行穷举。对于每种输入差分，输出差分的出现概率按其计数除以 2^{16} 计算。实验参数为

$$W = 8, \qquad M = \texttt{0x9}B, \qquad K = \texttt{0x46}.$$

代码对同一输入对使用相同的轮密钥 $K = \texttt{0x46}$。由于 K 在差分中抵消，轮密钥取值不会改变统计分布。程序分别统计输入差分 $(\texttt{0x00}, \texttt{0x01})$、$(\texttt{0x01}, \texttt{0x00})$、$(\texttt{0x00}, \texttt{0x02})$ 和 $(\texttt{0x02}, \texttt{0x01})$，并列出每种情况概率最高的五个输出差分。

8.3 实验结果

表中概率由完整的 2^{16} 个输入状态穷举得到；若多个输出差分的计数相同，则按输出差分数值升序排列。

Table 5: PHILOX-2W 一轮变换的高频输出差分

输入差分 $(\Delta L, \Delta R)$	排名	输出差分 $(\Delta L', \Delta R')$	计数	概率
$(0x00, 0x01)$	1	$(0x00, 0x9B)$	4096	0.0625000000
$(0x00, 0x01)$	2	$(0x00, 0xA7)$	4096	0.0625000000
$(0x00, 0x01)$	3	$(0x00, 0xAB)$	4096	0.0625000000
$(0x00, 0x01)$	4	$(0x01, 0x9B)$	2560	0.0390625000
$(0x00, 0x01)$	5	$(0x01, 0xA7)$	2560	0.0390625000
$(0x01, 0x00)$	1	$(0x00, 0x01)$	65536	1.0000000000
$(0x00, 0x02)$	1	$(0x01, 0x4A)$	4096	0.0625000000
$(0x00, 0x02)$	2	$(0x01, 0x36)$	3072	0.0468750000
$(0x00, 0x02)$	3	$(0x01, 0x4E)$	2560	0.0390625000
$(0x00, 0x02)$	4	$(0x02, 0xCA)$	2560	0.0390625000
$(0x00, 0x02)$	5	$(0x01, 0x3A)$	2048	0.0312500000
$(0x02, 0x01)$	1	$(0x00, 0x99)$	4096	0.0625000000
$(0x02, 0x01)$	2	$(0x00, 0xA5)$	4096	0.0625000000
$(0x02, 0x01)$	3	$(0x00, 0xA9)$	4096	0.0625000000
$(0x02, 0x01)$	4	$(0x01, 0x99)$	2560	0.0390625000
$(0x02, 0x01)$	5	$(0x01, 0xA5)$	2560	0.0390625000

当输入差分为 $(0x01, 0x00)$ 时，右半字输入差分为零，符合前述推导，输出差分始终为 $(0x00, 0x01)$。其余三种输入差分均会产生多个不同的输出差分，其中最高单项概率为 0.0625。

```
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab2\task5> uv run python .\test_philox_differential.py
W: 8
M: 0x9B
轮密钥 K: 0x46
每种输入差分遍历的输入状态数: 65536

输入差分 (00, 01)
不同输出差分数: 59
排名 | 输出差分 (ΔL', ΔR') | 计数 | 概率
   1 | (00, 9B)            |  4096 | 0.0625000000
   2 | (00, A7)            |  4096 | 0.0625000000
   3 | (00, AB)            |  4096 | 0.0625000000
   4 | (01, 9B)            |  2560 | 0.0390625000
   5 | (01, A7)            |  2560 | 0.0390625000
概率总和: 1.0000000000

输入差分 (01, 00)
不同输出差分数: 1
排名 | 输出差分 (ΔL', ΔR') | 计数 | 概率
   1 | (00, 01)            | 65536 | 1.0000000000
概率总和: 1.0000000000

输入差分 (00, 02)
不同输出差分数: 68
排名 | 输出差分 (ΔL', ΔR') | 计数 | 概率
   1 | (01, 4A)            |  4096 | 0.0625000000
   2 | (01, 36)            |  3072 | 0.0468750000
   3 | (01, 4E)            |  2560 | 0.0390625000
   4 | (02, CA)            |  2560 | 0.0390625000
   5 | (01, 3A)            |  2048 | 0.0312500000
概率总和: 1.0000000000

输入差分 (02, 01)
不同输出差分数: 59
排名 | 输出差分 (ΔL', ΔR') | 计数 | 概率
   1 | (00, 99)            |  4096 | 0.0625000000
   2 | (00, A5)            |  4096 | 0.0625000000
   3 | (00, A9)            |  4096 | 0.0625000000
   4 | (01, 99)            |  2560 | 0.0390625000
   5 | (01, A5)            |  2560 | 0.0390625000
概率总和: 1.0000000000
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab2\task5>
```

Figure 10: PHILOX-2W 一轮差分统计的程序运行结果

8.4 与随机置换的对比

随机置换程序先生成包含 0 至 $2^{16}-1$ 的列表，再使用 Python 标准库的洗牌操作将列表重排，得到一个固定种子生成的 16 比特置换。将输入 (L, R) 编码为

$$x = (L \ll W) \mid R,$$

然后通过置换表得到 $\pi(x)$，并将其拆分为两个 W 比特分支。程序对所有 2^{16} 个状态及同一组输入差分逐一计数。这里的置换作用于整个 16 比特状态，而不是分别对 L、R 两个分支置换。

随机置换只使用状态空间大小和输入差分，不使用 PHILOX 轮函数中的参数 M、K。参数 $W = 8$ 用于确定总状态宽度，并打包、拆分两个分支。随机置换使用固定种子 20261009，代码位于 `lab2/task5/compare_philox_random_permutation.py`。

对于非零输入差分，置换前后的两个状态必不相同，因此随机置换的输出差分不可能为零。对任一固定的非零输出差分 β，随机置换下的理论平均概率为

$$\frac{1}{2^{16}-1} \approx 0.0000152590.$$

单个随机置换的最高频率可能高于该平均值，这是从 $2^{16}-1$ 种非零输出差分中取最高项后的抽样波动。

图 11展示随机置换的运行结果。表 6汇总了两种映射在四种输入差分下的输出分布支持数及最高单项概率。随机置换的最高项均为 12 次，即 $12/2^{16} \approx 0.0001831$；PHILOX 的最高概率则为 0.0625，输入差分 $(\texttt{0x01}, \texttt{0x00})$ 时为 1。此外，随机置换的输出差分分散在约 2.57 万种取值中，而 PHILOX 在本实验中只出现 1 至 68 种输出差分，显示出明显不同的传播分布。

Table 6: PHILOX-2W 与随机置换的输出差分分布比较

输入差分 $(\Delta L, \Delta R)$	PHILOX 支持数	PHILOX 峰值概率	置换支持数	置换峰值概率
$(\texttt{0x00}, \texttt{0x01})$	59	0.0625000000	25796	0.0001831055
$(\texttt{0x01}, \texttt{0x00})$	1	1.0000000000	25824	0.0001831055
$(\texttt{0x00}, \texttt{0x02})$	68	0.0625000000	25725	0.0001831055
$(\texttt{0x02}, \texttt{0x01})$	59	0.0625000000	25789	0.0001831055

随机置换中具体哪些输出差分排在前列由种子决定；本次结果使用固定种子，便于复现。其单次最高项高于理论平均概率 $1/(2^{16}-1)$，是从大量可能的非零输出差分中取最大值所产生的波动。

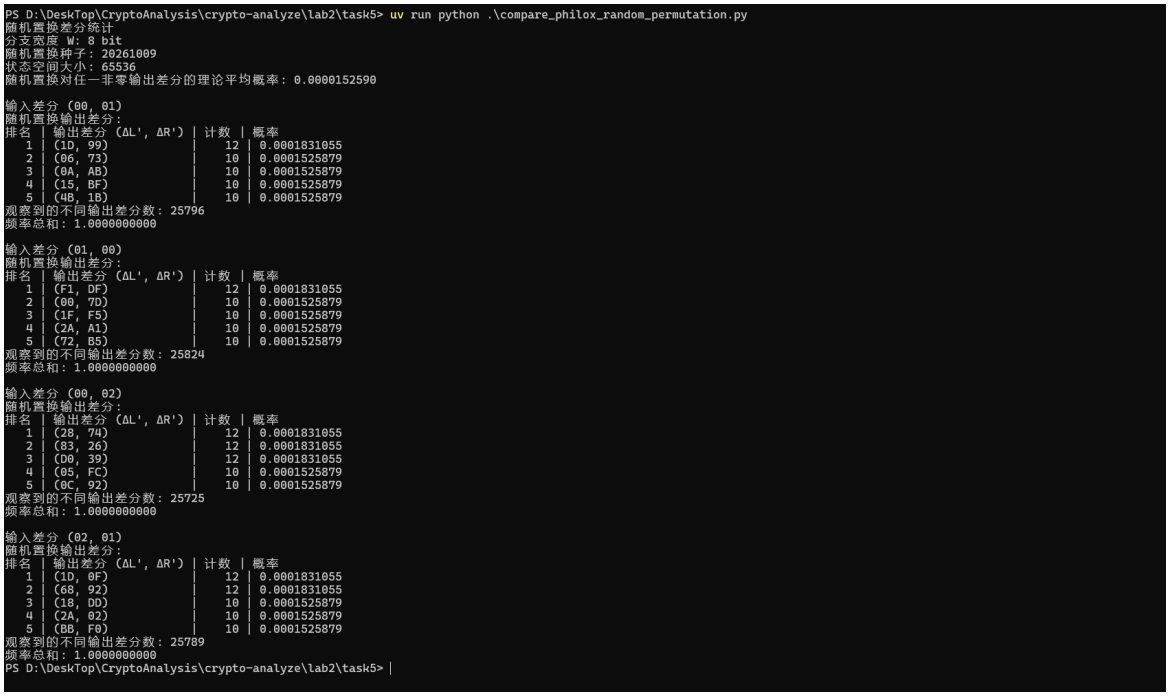


Figure 11: 随机置换差分统计及与 PHILOX-2W 的运行结果对照

9 实验总结

本实验利用 S 盒差分分布表搜索四轮高概率差分及其路线，并通过多组密钥下的穷举结果进行验证，观察到实验频率与理论概率接近。任务四使用该差分筛选明密文对并对最后一轮的 4 比特子密钥进行评分，正确值进入最高分候选，但本次结果未能唯一确定密钥。任务五穷举 PHILOX-2W 一轮差分并与相同状态空间的随机置换比较，PHILOX 的输出差分分布更集中。通过这些实验，完成了从差分搜索、实验验证到部分密钥恢复和置换对照的分析过程。

A 实验代码

见代码附录文件。

B 实验报告完整地址

实验完整代码、实验报告 LaTeX 源代码见仓库：https://github.com/infinitepwn/crypto-analyze.git

C AI 使用情况

本实验使用 AI 辅助编程和实验报告的排版、错别字检查，实验主体流程设计、结果分析和实验报告的撰写未使用 AI。